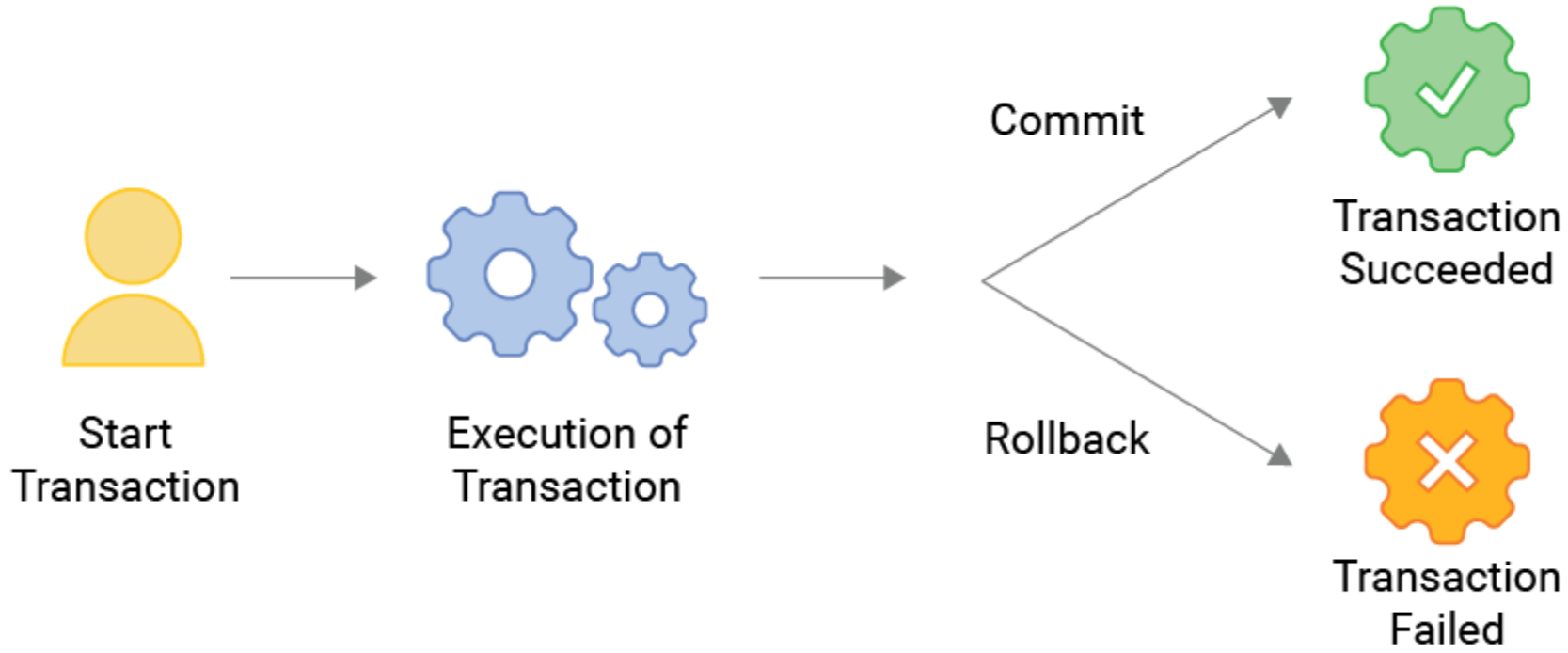
Transaction Processing Concepts

Introduction

- Transaction
  - Describes local unit of database processing
- Transaction processing systems
  - Systems with large databases and hundreds of concurrent users
  - Require high availability and fast response time

Transactions, Read and Write Operations, and DBMS Buffers

- **A Transaction** : is a logical unit of database processing that includes one or more database access operation.
- All database access operations between **Begin Transaction** and **End Transaction** statements are considered one logical transaction.
- If the database operations in a transaction do not update the database but only retrieve data , the transaction is called a **read-only transaction.**
- **Basic database access operations** :
  - **read_item(X)** : reads a database item **X** into program variable.
  - **Write_item(X)** : Writes the value of program variable **X** into the database item **X**.


Commit
Transaction Succeeded
Start Transaction
Execution of Transaction
Rollback
Transaction Failed

- **Read(X)**
- A **read operation** is used to read the value of **X** from the database and store it in a buffer in the main memory for further actions such as displaying that value.
- **Write(X)**
- A write operation is used to write the value to the database from the buffer in the main memory.

- **Commit**
- This operation in transactions is used to maintain integrity in the database. Due to some failure of power, hardware, or software, etc., a transaction might get interrupted before all its operations are completed
- **Rollback**
- This operation is performed to bring the database to the last saved state when any transaction is interrupted in between due to any power, hardware, or software failure.

Transactions

- A **transaction** is a *unit* of program execution that accesses and possibly updates various data items.
- E.g., transaction to transfer $50 from account A to account B:
  1. **read**(*A*)
  2. *A* := *A* – 50
  3. **write**(*A*)
  4. **read**(*B*)
  5. *B* := *B* + 50
  6. **write**(*B*)
- Multiple transactions can run at the same time

Introduction to Transaction Processing

- Single-user DBMS
  - At most one user at a time can use the system
  - Example: home computer
- Multiuser DBMS
  - Many users can access the system (database) concurrently
  - Example: airline reservations system

Multiple Transactions

- Two modes of execution
  - Concurrent execution
  - Parallel execution
- What is the difference between the two modes of executions?

Transactions

- Transaction: an executing program
  - Forms logical unit of database processing
- Begin and end transaction statements
  - Specify transaction boundaries
- Read-only transaction
- Read-write transaction

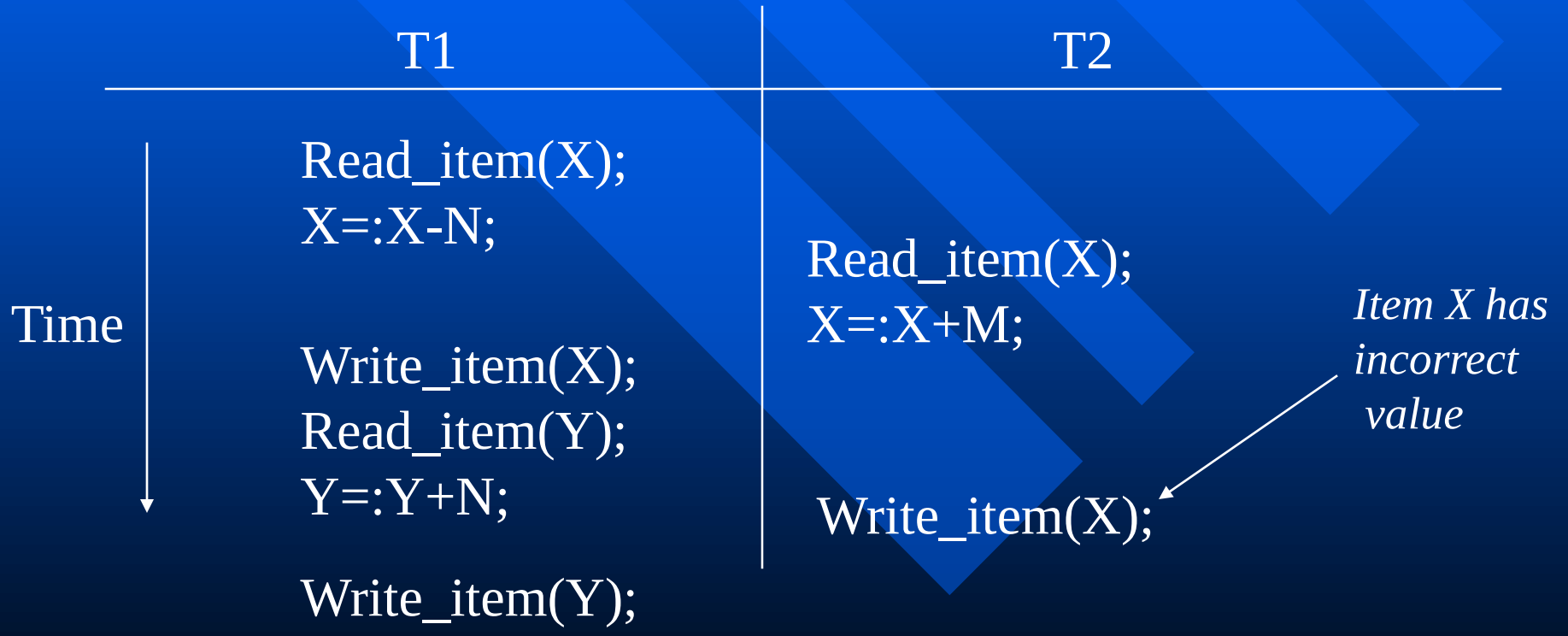
Concurrency Control

- Transactions submitted by various users may execute concurrently
  - Access and update the same database items
  - Some form of concurrency control is needed
- The lost update problem
  - Occurs when two transactions that access the same database items have operations interleaved
  - Results in incorrect value of some database items

Why Concurrency Control is needed

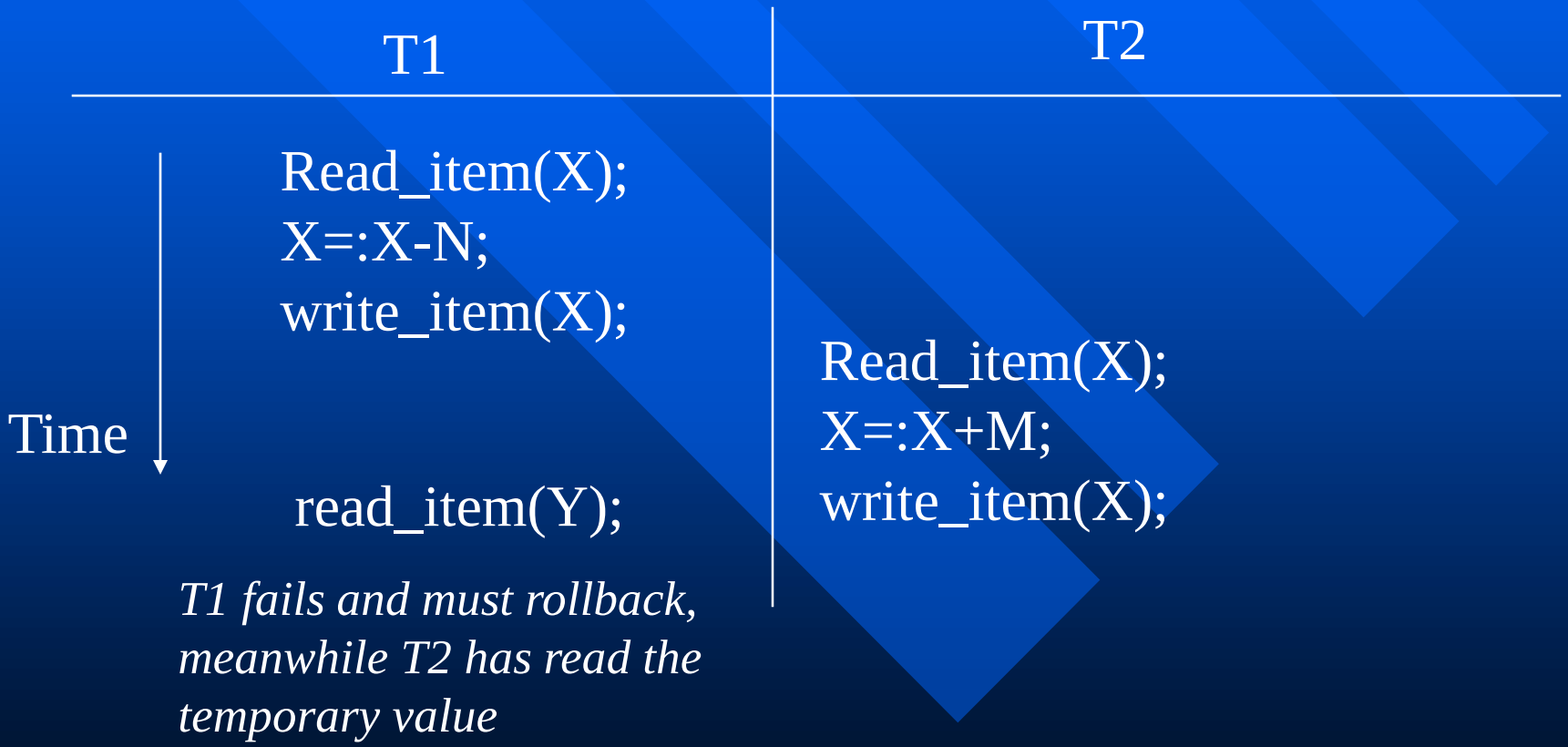
- **Several problems can occur when concurrent transactions execute in an uncontrolled manner.**

1. **The Lost Update Problem :**



Why Concurrency Control is needed (continued)

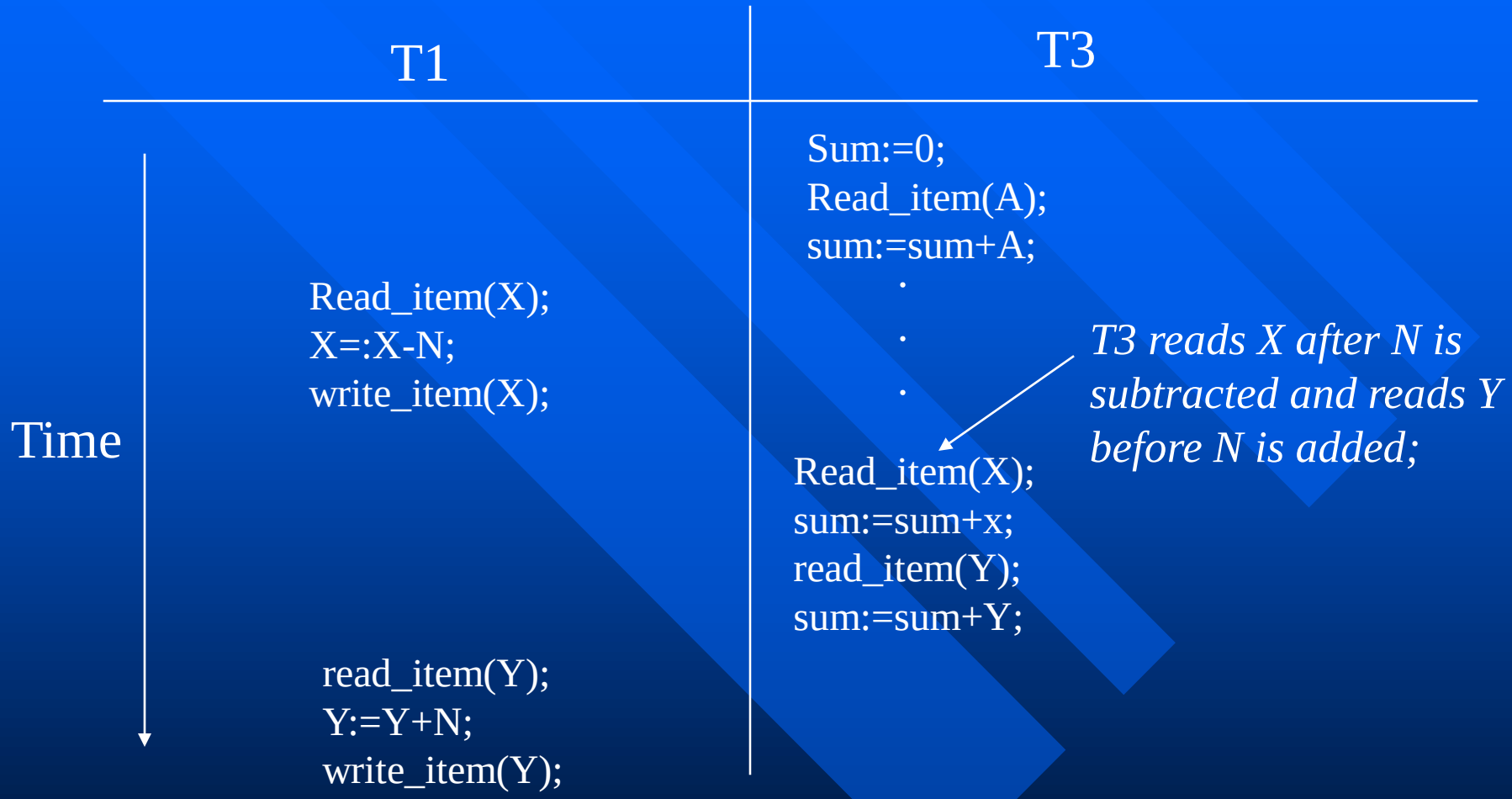
2. The temporary Update (or Dirty read) problem



T1 fails and must rollback, meanwhile T2 has read the temporary value

Why Concurrency Control is needed (continued)

3. The Incorrect Summary Problem



4. Unrepeatable Read problem: **A transaction reads items twice with two different values because it was changed by another transaction between the two reads.**

The Unrepeatable Read Problem

- Transaction T reads the same item twice
- Value is changed by another transaction T′ between the two reads
- T receives different values for the two reads of the same item

Transaction and System Concepts (cont'd.)

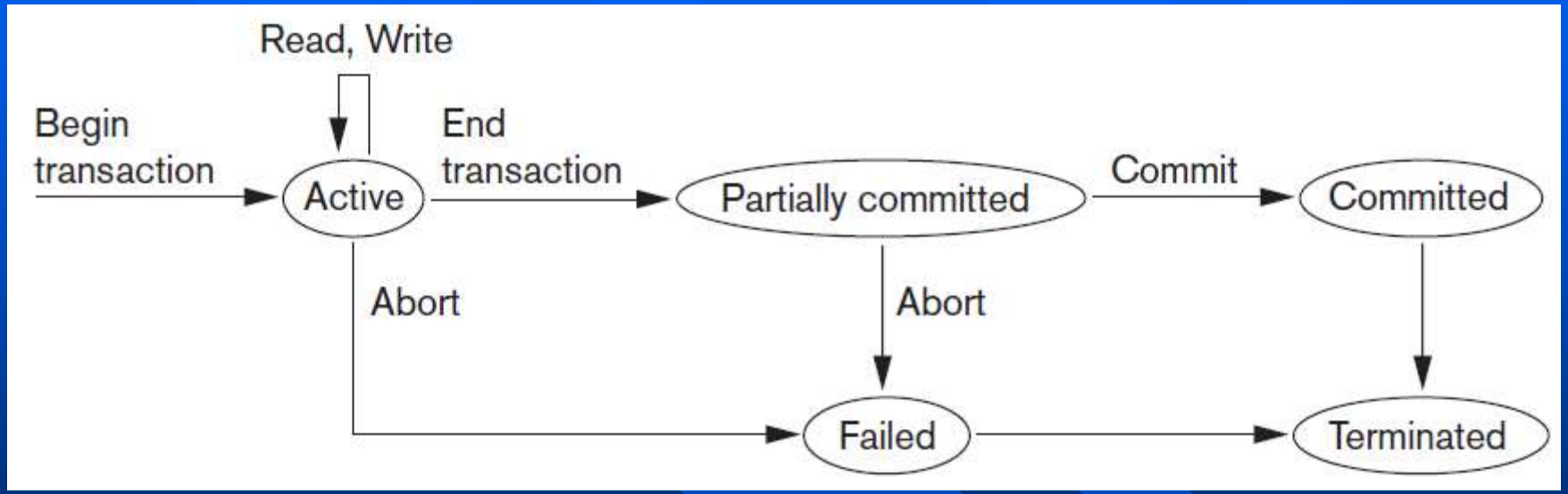


Figure 20.4 State transition diagram illustrating the states for transaction execution

Why Recovery Is Needed

- **There are several possible reasons for a transaction to fail**
  - ***A computer failure*** : A hardware, software, or network error occurs in the computer system during transaction execution.
  - ***A transaction or system error*** : Some operations in the transaction may cause it to fail.
  - **Local errors or exception conditions** detected by the transaction.
  - ***Concurrency control enforcement*** : The concurrency control method may decide to abort the transaction.
  - ***Disk failure*** : all disk or some disk blocks may lose their data.
  - ***Physical problems*** : Disasters, theft, fire, etc.

Desirable Properties of transactions

- **ACID should be enforced by the concurrency control and recovery methods of the DBMS.**
- **ACID properties of transactions :**
  - ***Atomicity*** : a transaction is an atomic unit of processing; it is either performed entirely or not performed at all.

    (It is the responsibility of recovery)

  - ***Consistency*** : transfer the database from one consistent state to another consistent state

    (It is the responsibility of the applications and DBMS to maintain the constraints)

Desirable Properties of transactions (continued)

– ***Isolation*** : the execution of the transaction should be isolated from other transactions (Locking)

(It is the responsibility of concurrency)

* Isolation level:

-Level 0 (no dirty read) **-Level 1 (no lost update)**

-Level 2 (no dirty+ no lost) **-Level 3 (level 2+repeatable reads)**

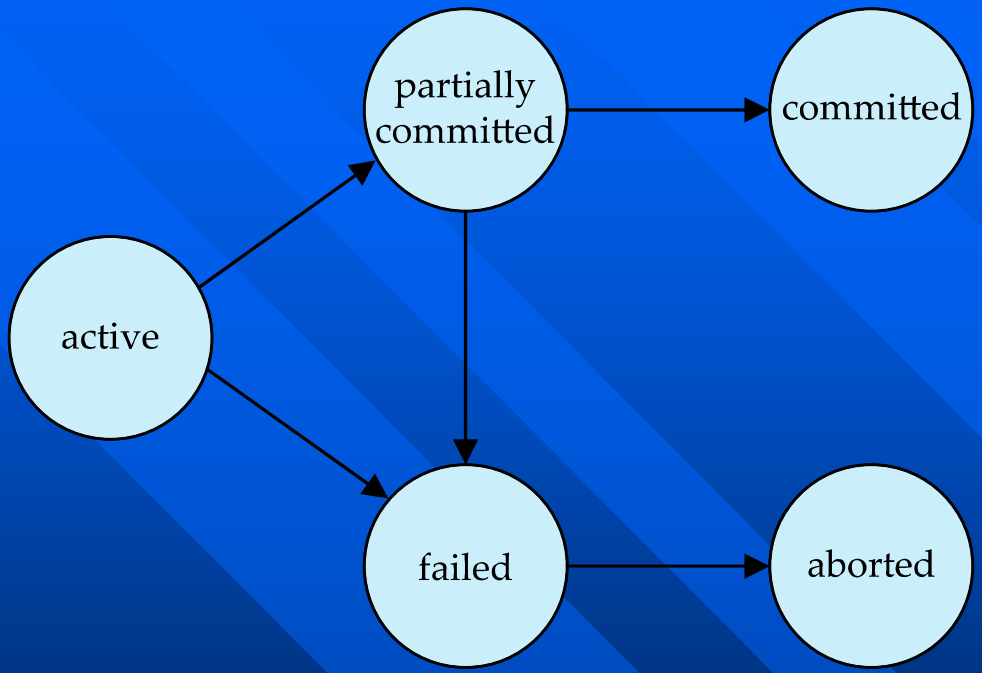
– ***Durability*** : committed transactions must persist in the database ,i.e. those changes must not be lost because of any failure.

(It is the responsibility of recovery)

Transaction State

- **Active** – the initial state; the transaction stays in this state while it is executing
- **Partially committed** – after the final statement has been executed.
- **Failed** -- after the discovery that normal execution can no longer proceed.
- **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
  - Restart the transaction
    - » Can be done only if no internal logical error caused the transaction to abort
    - » Example of such an error?
  - Kill (abort) the transaction
- **Committed** – after successful completion.

Transaction State (Cont.)



Schedules of Transactions

- Schedule(or History) S **of n transactions $T_1, T_2, .., T_n$ is an ordering of operations of the transactions with the following conditions :**
  - **for each transaction T_i that participate in S , the operations of T_i in S must appear in the same order in which they occur in T_i**
  - **the operations from other transactions T_j can be interleaved with the operations of T_i in S.**
- **The symbols r,w,c, and a are used for the operations read_item, write_item, commit, and abort respectively.**

Schedule 1

- Let T_1 transfer \$50 from A to B, and T_2 transfer 10% of the balance from A to B.
- A **serial** schedule in which T_1 is followed by T_2 :

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

Schedule 2

- A serial schedule where T_2 is followed by T_1

T_1	T_2
	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	

Schedule 3

- Let T_1 and T_2 be the transactions defined previously. The following schedule is not a serial schedule, but it is ***equivalent*** to Schedule 1

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (B)
	$B := B + temp$
	write (B)
	commit

Schedule 4

- The following concurrent schedule does not preserve the sum of (*A* + *B*).

T_1	T_2
read (*A*)	
A := *A* – 50	
	read (*A*)
	temp := *A* * 0.1
	A := *A* - *temp*
	write (*A*)
	read (*B*)
write (*A*)	
read (*B*)	
B := *B* + 50	
write (*B*)	
commit	
	B := *B* + *temp*
	write (*B*)
	commit

Schedules of Transactions (continued)

- **Two operations are said to be conflict if:**
  - they belong to different tansactions
  - they access the same item X
  - at least one the operations is a write_item(X)

Ex:

S1: $r_1(x);r_2(x);w_1(x);r_1(y);w_2(x);w_1(y);$

conflicts: $[r_1(x);w_2(x)] - [r_2(x);w_1(x)] - [w_1(x); w_2(x)]$

Characterizing Schedules based on Recoverability

- Type of schedules :
  - ***recoverable scheduls*** : once a transaction T is commited , it should never rollbacked.

    » **The condition of recoverable schedule**:
    - **No transaction T in S commits until all transactions T' that have written an item that T reads have committed.**

    » It is possible for Cascading rollback to occur when an uncommited transaction has to be rollbacked.

Examples

- 1- S_a: $r_1(x);r_2(x);w_1(x);r_1(y);w_2(x);c_2;w_1(y);c_1;$

  Recoverable schedule, even it suffers from the lost update problem [$w_1(x);w_2(x)$]

- 2- S_c: $r_1(x);\ w_1(x);\ r_2(x);\ r_1(y);w_2(x);c_2;a_1;$

  Nonrecoverable schedule. Why?

  T_2 reads x from T_1, and then T_2 commits before T_1 commits. If T_1 aborts, then T_2 must be aborted after had been committed.

Examples

- 3- To make S_2 recoverable, c_2 of S_2 must be postponed until after T_1 commits as follows:

  S_d: $r_1(x)$; $w_1(x)$; $r_2(x)$; $r_1(y)$;$w_2(x)$; $w_1(y)$;c_1; c_2;

- If T_1 aborts, then T_2 should also abort as follows:

  S_e: $r_1(x)$; $w_1(x)$; $r_2(x)$; $r_1(y)$;$w_2(x)$; $w_1(y)$;a_1; a_2;